

STEP 2 — Import Required Libraries

```
import nltk
import string
import math
import random
from collections import Counter, defaultdict

import numpy as np
import pandas as pd
```

STEP 3 : Load the dataset

```
nltk.download('gutenberg')
from nltk.corpus import gutenberg

# Load text corpus
text = gutenberg.raw('austen-emma.txt')

# Display sample text
print(text[:500])
```

```
[nltk_data] Downloading package gutenberg to /root/nltk_data...
[nltk_data] Unzipping corpora/gutenberg.zip.
[Emma by Jane Austen 1816]
```

VOLUME I

CHAPTER I

Emma Woodhouse, handsome, clever, and rich, with a comfortable home and happy disposition, seemed to unite some of the best blessings of existence; and had lived nearly twenty-one years in the world with very little to distress or vex her.

She was the youngest of the two daughters of a most affectionate, indulgent father; and had, in consequence of her sister's marriage, been mistress of his house from a very early period. Her mother had died t

```
words = text.split()
print("Total number of words:", len(words))
```

Total number of words: 158167

```
split_point = int(0.8 * len(words))

train_words = words[:split_point]
test_words = words[split_point:]

print("Training words:", len(train_words))
print("Testing words:", len(test_words))
```

```
Training words: 126533
Testing words: 31634
```

STEP 4 — Preprocess Text

```
nltk.download('punkt')
nltk.download('stopwords')

from nltk.tokenize import wordpunct_tokenize
from nltk.corpus import stopwords
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Unzipping corpora/stopwords.zip.
```

```
stop_words = set(stopwords.words('english'))

def preprocess_text(words):
```

```

cleaned_words = []
for word in words:
    word = word.lower()
    word = word.translate(str.maketrans("", "", string.punctuation))
    if word.isalpha():
        cleaned_words.append(word)
return cleaned_words

```

```

train_clean = preprocess_text(train_words)
test_clean = preprocess_text(test_words)

```

```

def add_sentence_tokens(words):
    return ["<s>"] + words + ["</s>"]

train_clean = add_sentence_tokens(train_clean)
test_clean = add_sentence_tokens(test_clean)

```

```
train_clean[:20]
```

```

['<s>',
 'emma',
 'by',
 'jane',
 'austen',
 'volume',
 'i',
 'chapter',
 'i',
 'emma',
 'woodhouse',
 'handsome',
 'clever',
 'and',
 'rich',
 'with',
 'a',
 'comfortable',
 'home',
 'and']

```

STEP-5 — Build N-Gram Models

```

from collections import Counter

# Unigram counts
unigram_counts = Counter(train_clean)

# Total number of words
total_unigrams = sum(unigram_counts.values())

# Unigram probabilities
unigram_probs = {word: count / total_unigrams
                  for word, count in unigram_counts.items()}

# View sample
list(unigram_counts.items())[:10]

```

```

[('<s>', 1),
 ('emma', 573),
 ('by', 433),
 ('jane', 225),
 ('austen', 1),
 ('volume', 3),
 ('i', 2389),
 ('chapter', 46),
 ('woodhouse', 221),
 ('handsome', 34)]

```

```

# Create bigrams
bigrams = [(train_clean[i], train_clean[i+1])
            for i in range(len(train_clean)-1)]

# Bigram counts
bigram_counts = Counter(bigrams)

# Bigram probabilities

```

```

bigram_probs = {}
for (w1, w2), count in bigram_counts.items():
    bigram_probs[(w1, w2)] = count / unigram_counts[w1]

# View sample
list(bigram_counts.items())[:10]

```

```

[ (('<s>', 'emma'), 1),
  (('emma', 'by'), 1),
  (('by', 'jane'), 1),
  (('jane', 'austen'), 1),
  (('austen', 'volume'), 1),
  (('volume', 'i'), 1),
  (('i', 'chapter'), 1),
  (('chapter', 'i'), 3),
  (('i', 'emma'), 3),
  (('emma', 'woodhouse'), 2)]

```

```

# Create trigrams
trigrams = [(train_clean[i], train_clean[i+1], train_clean[i+2])
             for i in range(len(train_clean)-2)]

# Trigram counts
trigram_counts = Counter(trigrams)

# Trigram probabilities
trigram_probs = {}
for (w1, w2, w3), count in trigram_counts.items():
    trigram_probs[(w1, w2, w3)] = count / bigram_counts[(w1, w2)]

# View sample
list(trigram_counts.items())[:10]

```

```

[ (('<s>', 'emma', 'by'), 1),
  (('emma', 'by', 'jane'), 1),
  (('by', 'jane', 'austen'), 1),
  (('jane', 'austen', 'volume'), 1),
  (('austen', 'volume', 'i'), 1),
  (('volume', 'i', 'chapter'), 1),
  (('i', 'chapter', 'i'), 1),
  (('chapter', 'i', 'emma'), 2),
  (('i', 'emma', 'woodhouse'), 1),
  (('emma', 'woodhouse', 'handsome'), 1)]

```

STEP-6: Apply Smoothing

```
vocab_size = len(unigram_counts)
```

```

def unigram_prob_laplace(word):
    return (unigram_counts.get(word, 0) + 1) / (total_unigrams + vocab_size)

```

```

def bigram_prob_laplace(w1, w2):
    return (bigram_counts.get((w1, w2), 0) + 1) / (unigram_counts.get(w1, 0) + vocab_size)

```

```

def trigram_prob_laplace(w1, w2, w3):
    return (trigram_counts.get((w1, w2, w3), 0) + 1) / (bigram_counts.get((w1, w2), 0) + vocab_size)

```

```

print(unigram_prob_laplace("emma"))
print(bigram_prob_laplace("emma", "was"))
print(trigram_prob_laplace("emma", "was", "a"))

```

```

0.004264614104431038
0.006221914967162115
0.00024512807942149775

```

STEP-7: Sentence Probability Calculation

```

sentences = [
    "emma was a clever girl",
    "the movie was very good",
    "she enjoyed the book",

```

```
"this sentence is completely new",
"emma liked the story"
]
```

```
def preprocess_sentence(sentence):
    tokens = sentence.lower().split()
    tokens = ["<s>"] + tokens + ["</s>"]
    return tokens

processed_sentences = [preprocess_sentence(s) for s in sentences]
processed_sentences
```

```
[['<s>', 'emma', 'was', 'a', 'clever', 'girl', '</s>'],
['<s>', 'the', 'movie', 'was', 'very', 'good', '</s>'],
['<s>', 'she', 'enjoyed', 'the', 'book', '</s>'],
['<s>', 'this', 'sentence', 'is', 'completely', 'new', '</s>'],
['<s>', 'emma', 'liked', 'the', 'story', '</s>']]
```

```
import math

def sentence_prob_unigram(sentence):
    prob = 1
    for word in sentence:
        prob *= unigram_prob_laplace(word)
    return prob

for s in processed_sentences:
    print(sentence_prob_unigram(s))
```

```
1.2577896567780522e-23
9.259656065171087e-24
2.6044979187800407e-22
4.019664008983234e-27
2.1405810500855434e-22
```

```
def sentence_prob_bigram(sentence):
    prob = 1
    for i in range(len(sentence)-1):
        prob *= bigram_prob_laplace(sentence[i], sentence[i+1])
    return prob

for s in processed_sentences:
    print(sentence_prob_bigram(s))
```

```
9.990294853997825e-20
4.182105588795599e-21
4.674338147709711e-19
2.9553326432442586e-24
7.066231575528645e-19
```

```
def sentence_prob_trigram(sentence):
    prob = 1
    for i in range(len(sentence)-2):
        prob *= trigram_prob_laplace(sentence[i], sentence[i+1], sentence[i+2])
    return prob

for s in processed_sentences:
    print(sentence_prob_trigram(s))
```

```
1.119498610040482e-19
2.822105469476785e-20
4.628375958242797e-16
2.857373436277869e-20
4.628375887820807e-16
```

STEP-8: Perplexity Calculation

```
import math

def perplexity_unigram(sentence):
    log_prob = 0
    N = len(sentence)
    for word in sentence:
        log_prob += math.log(unigram_prob_laplace(word))
    return math.exp(-log_prob / N)
```

```
def perplexity_bigram(sentence):
    log_prob = 0
    N = len(sentence) - 1
    for i in range(len(sentence)-1):
        log_prob += math.log(bigram_prob_laplace(sentence[i], sentence[i+1]))
    return math.exp(-log_prob / N)
```

```
def perplexity_trigram(sentence):
    log_prob = 0
    N = len(sentence) - 2
    for i in range(len(sentence)-2):
        log_prob += math.log(trigram_prob_laplace(sentence[i], sentence[i+1], sentence[i+2]))
    return math.exp(-log_prob / N)
```

```
for s in processed_sentences:
    print("Sentence:", " ".join(s))
    print("Unigram Perplexity:", perplexity_unigram(s))
    print("Bigram Perplexity:", perplexity_bigram(s))
    print("Trigram Perplexity:", perplexity_trigram(s))
    print("-" * 40)
```

```
Sentence: <s> emma was a clever girl </s>
Unigram Perplexity: 1868.4632860854733
Bigram Perplexity: 1468.0368222310697
Trigram Perplexity: 6168.7232714953025
-----
Sentence: <s> the movie was very good </s>
Unigram Perplexity: 1952.0298259701665
Bigram Perplexity: 2491.3466933627706
Trigram Perplexity: 8126.159678107589
-----
Sentence: <s> she enjoyed the book </s>
Unigram Perplexity: 3957.12100588124
Bigram Perplexity: 4635.066556158923
Trigram Perplexity: 6817.77797340869
-----
Sentence: <s> this sentence is completely new </s>
Unigram Perplexity: 5899.701452634722
Bigram Perplexity: 8347.67639400177
Trigram Perplexity: 8105.999999999997
-----
Sentence: <s> emma liked the story </s>
Unigram Perplexity: 4088.632287837651
Bigram Perplexity: 4267.390900137016
Trigram Perplexity: 6817.777823274453
-----
```

STEP-9: Comparison and Analysis

In this experiment, Unigram, Bigram, and Trigram language models were compared using perplexity. The Bigram and Trigram models generally produced lower perplexity values than the Unigram model, indicating better prediction capability. The Unigram model performs poorly because it ignores word order and context. Bigram models capture local context by considering the previous word, which improves performance. Trigram models capture even more context, but they sometimes suffer from data sparsity. Trigrams did not always perform best because many word combinations were unseen in the training data. When unseen words or N-grams appear, the model assigns very low probabilities even with smoothing. Laplace smoothing reduced zero-probability issues and made all models more robust. Overall, the Bigram model provided a good balance between context and data availability.

